ECON 370: Assignment 2

Jiaxi Li

Due: Before class on October 2nd, 2025

Names of all group members:

Directions

As a reminder, LLMs and AIs are strictly forbidden from use on this assignment.

Use the Rmd template and write code to answer the following questions. In order to ensure smooth grading, I will ask you to write both your code and explanation (as comment) in code chunk. Please write your code and answer in the corresponding place.

Each question is worth three points. For each section, a "question" corresponds to a number.

Please **comment** your code clearly. 0.5 point per question will be deducted if there is a significantly lack of commenting. Also, the easier your code is to read, the more likely I will be able to figure out what you're doing.

For questions creating functions in this assignment, you must have a "**return**". Otherwise, 0.5 point per question will be deducted. For any question involving a random number generation, you must **set seed**. Otherwise, 0.5 point per question will be deducted. For questions creating loops in this assignment, you must **preallocate**. Otherwise, 0.5 point per question will be deducted.

You may work in groups of three to four people for this homework. Please submit one script per group and clearly indicate who was in your group for the assignment.

O. Data

This is the code to extract data online. Do not modify.

I. Logic

In class, we have used stock prices to calculate the stock returns. In this section. we will work with price information instead. Create subsets of the "SP500_monthly_df" data (provided in section O) using the following conditions:

1. The months that the low price is less or equal to 2000. Call this subset "SP500_monthly_df_low2000":

2. The months that the close price is greater than the open price. Call this subset "SP500_monthly_df_cgo":

3. The months that the adjusted close price ("adjusted") is different from the close price. Call this subset "SP500_monthly_df_adj":

4. The months that the trading volume ("volume") is between 3500000000 and 4000000000 (including equal to 3500000000 and 4000000000). Call this subset "SP500_monthly_df_midvol":

5. The months that with specific dates "2018-01-31" and "2018-12-31". Call this subset "SP500_monthly_df_endbegin2018

II. Functions

1. Create your own absolute value function that takes in a single value and returns it's absolute value. Call it "my_abs". Note: ***Do not use the "abs()" function.***

2. Now, create your own absolute value function that takes in *a vector* and returns it's absolute value. Call it "my_abs_vec". Note: ***Do not use the "abs()" function.***

3. Create a function that returns "the sign" of a single number. That is, if the number is positive, return 1, if the number is negative, return -1, and if the number if 0, return 0. Call this function "my_sign". Note: ***Do not use the "sign()" function.***

4. Create a function that returns the sign of a vector of numbers. Call this function "my_sign_vec". Note: ***Do not use the "sign()" function.***

5. Write a function called CRRA that takes in two inputs: a vector v and η (spelled eta) and returns

$$u(v;\eta) = \begin{cases} \dfrac{v^{1-\eta}-1}{1-\eta}, & \eta \geq 0 \text{ and } \eta \neq 1 \\ \log(v), & \eta = 1 \end{cases}$$

Note, if $\eta < 0$, the function should error out. Likewise, you should write this function to be able to take in ***a vector of length greater than one*** for v but ***only one value*** of η.

6. Create a function called "my_funct" that calculates the following:

$$f(x) = \begin{cases} x^2 + 2x + |x|, & x < 0 \\ x^2 + 3 + \log(x+1), & 0 \leq x < 2 \\ x^2 + 4x - 14, & x \geq 2 \end{cases}$$

   Note: This function should be able to take a vector for x.

7. Create the following object and calculate the mean, median, min, max and standard deviation of each row and column. Name each output "x_y" where x is the statistic that is being calculated and y is either row or column. (for example, mean_row and mean_column.)

III. Loops, Apply, future.apply and Vectorization

1. Remember that we have talked about the definition of Fibonacci sequence in class. Use loop to generate the **first 20** Fibonacci sequence, starting from $X_1 = 0$ and $X_2 = 1$: $X_{n+2} = X_{n+1} + X_n$ for $n \geq 1$.

2. Can we vectorize the above operation? Provide the code if yes. Provide explanation if no.

3. Can we use apply functions for the above operation? Provide the code if yes. Provide explanation if no. (Hint: you can use a combination of functions and force assignment in the function. Note that the whole purpose of the function is to update, so no "return" here.)

4. Can we use future.apply functions for the above operation? Provide the code if yes (use 4 cores if possible). Provide explanation if no.

5. Use loop to calculate the mean for 10 random numbers **100 times**. (Remember to set seeds before the operation. Use "rnorm" to generate the random numbers. **Do not save those means but use "print" to show them.**)

6. Can we vectorize the above operation? Provide the code if yes. Provide explanation if no. (Hint: you can use a matrix to do "vectorization" here.)

7. Can we use apply functions for the above operation? Provide the code if yes. Provide explanation if no.

8. Can we use future.apply functions for the above operation? Provide the code if yes (use 4 cores if possible). Provide explanation if no.

IV. Advanced Loops

1. Use the loop we do in class for $\sqrt{X}$. Add "**cat**" function to indicate every 10 iterations. "**break**" and "**warning**" if we reach 50 iterations.

2. Note that we can use the "**stop**" function instead. What would be the difference between "break" + "warning" vs. "stop" when we reach 100 iterations? (Do not need to code here. Just talk about the difference.)

3. Return indicates the percentage growth of wealth in one period. The cumulative gross return keeps track of the cumulative wealth growth rate over time. Given the returns as $r_1, r_2, \ldots, r_k$, the cumulative gross return is defined as $CR_0 = 1$, $CR_1 = (1 + r_1)$, $CR_2 = CR_1(1 + r_2)$, $CR_k = CR_{k-1}(1 + r_k)$.

   Use the "SP500_monthly_df" data (provided in section O) and appropriate loop to estimate the cumulative return. Make sure that it is saved in a new column in "SP500_monthly_df" called "CR". Use "message" to print "Congratulations, you have doubled your wealth!" when the cumulative wealth growth rate first gets above 2. (Be creative about it!)

4. Can we vectorize this operation? Provide the code if yes. Provide explanation if no.

   Also, use "ggplot" to plot the cumulative return against date. Make sure to have the axis labeled and have a title. The y-axis needs to be labeled "Cumulative Gross Return". The title needs to be "Cumulative Gross Return of the SP500 Index". And the title must be centered in the middle.

5. Create a function called "**safe_factorial**". When the input is negative, "`stop`" with "Input is negative!"; when the input is not negative, calculate factorial using available "**factorial**" function. Create a loop to take factorial of values starting at -49 and ending at 50, with increments of 1. What happens? **Do not save the results, just run safe_factorial in the loop.**

6. Use your "**safe_factorial**" for factorial function and "**tryCatch**" to prevent the loop from stopping. Print a **message** when it is failing "Fail to calculate factorial at the value of **xxx**", where **xxx** is the value. **Do not save the results, just run safe_factorial with tryCatch in the loop.**